Algorithm Problems

Nathan Gremillion

March 7, 2026

Chapter 2 (Getting Started)

Section 2.1 (Insertion sort)

2.1-4
Consider the searching problem:
Input: A sequence of n numbers $< a_1, a_2, ..., a_n >$ stored in array A[1:n] and a value x.
Output: An index i that x equals A[i] or the special value *NIL* if x doesn't appear in A.

Write pseudocode for linear search, which scans through the array from begin-ning to end, looking for x. Using a loop invariant, prove that your algorithm is correct. Make sure that your loop invariant fulûlls the three necessary properties.

```
# give the index of the x target or else -1 if not found
def find_val(x: int, array: list[int]) -> int:
    n: int = len(array)
    res: int = -1

    for i in range(n):
        if (array[i] == x):
            res = i
            break

    return res

array: list[int] = [1,2,3,4,5,6,7]
index: int = find_val(100, array)

print(f"Index: {index}")
```

Note: Loop invariant properites include:

- **Initialization**: The invariant is true prior to the first iteration of the loop.
- **Maintenance**: If the invariant is true before an iteration of the loop, it remains true before the next iteration.
- **Termination**: The loop eventually terminates. When it does, the invariant—together with the reason the loop terminated—gives a useful property that helps prove the algorithm is correct.

Proof. **Initialization:** Before the first iteration, the loop variable i is set to 0. At this point, the subarray $A[0:i]$ contains no elements. Therefore, no index $j < i$ exists such that $A[j] = x$. The loop invariant holds because we have not yet examined any elements, so it is correct that x has not been found in the subarray. Thus we return Nil (-1) which is a valid solution in this scenerio.

Maintenance: Next, we show that each iteration maintains the loop invariant. This means that if the invariant holds for the nth iteration, then it also holds for the (n+1)th iteration

Assume that at the start of the nth iteration the invariant holds. That is, we have already checked all indices $i \in [0, n-1]$ and found that $A[i] \neq x$. Therefore the value x has not been found in the first n element.

During the next iteration we check at index n. If A[n] == x then the algorithm returns the index n. Otherwise we continue to the next iteration and we know that A[0:n] does not conatin x

Thus loop invariant holds for the next iteration. And thus we have shown Maintenance holds for the algorithm.

Termination The loop terminates when either the element x is found or when the index i reaches n, the lengthe of the array. Since the loop increments i each iteration and $0 \leq i \leq n$, the algorithm must terminate.

If the algorithm terminate because A[i] == x, then the it correctly returns the index i.

If the loop terminates because i == n, the by the loop invariant we know that x is not in A[0:n-1]. Therefore, the element x does not appear in the array, and we return -1 indicating this. □

2.1-5

Consider the problem of adding two n-bit binary integers a and b, stored in two n-element arrays A[0:n-1] and B[0:n-1], where each element is either 0 or 1, a = $\sum_{i=0}^{n-1}$ A[i]*2^i, and b = $\sum_{i=0}^{n-1} B[i] * 2^i$. The sum c = a + b of the two integers should be stored in binary form in an (n + 1)-element array C[0:n], where c = $\sum_{i=0}^{n} C[i] * 2^i$. Write a procedure ADD-BINARY-INTEGERS that takes as input arrays A and B, along with the length n, and returns array C holding the sum.

```
def add_binary_arrays(binary1: list[str], binary2: list[str]) -> list[str]:
# keep pointers for binary 1 and binary 2
# from the question they have the same len so
# it is fair to say len(binary1) == len(binary2) == n
n: int = len(binary1)
i: int = n-1
carry: bool = False
C: list[str] = ['0'] * (n + 1)


while (i >= 0):
    bit1: str = binary1[i]
    bit2: str = binary2[i]
```

```
        if (bit1 == '1' and bit2 == '1'):
            if (carry):
                C[i+1] = '1'
            carry = True

        elif ((bit1 == '1' and bit2 == '0')
            or (bit1 == '0' and bit2 == '1')):
            if (not carry):
                C[i+1] = '1'
                carry = False
            else:
                carry = True

        else: # both are 0
            if (carry):
                C[i+1] = '1'
            carry = False

        i -= 1

    # check carry for the last bit
    if (carry):
        C[0] = '1'

    return C
```

Section 2.2 (Analyzing algorithms)

2.2-1
Express the function $n^3/1000 + 100n^2 - 100n + 3$ in terms of θ-notation.

So we have $c_1n^3 + c_2n^2 - c_3n$ we know the we can cancel out the constant terms since they are negligable for large vlaues of n and we know n^3 grows the fastest since it is a cubic. $\theta(n^3)$ is the answer

2.2-2
Consider sorting n numbers stored in array A[1:n] by first finding the smallest element of A[1:n] and exchanging it with the element in A[1]. Then find the smalles element of A[2:n], and exchange it with A[2]. Then find the smallest element of A[3:n], and exchange it with A[3]. Continue this for the first n-1 elements of A. Write the code for this algorithm (this is selection sort). What loop invariant does this algorithm maintain? Why does it need to run for only the first n-1 elements, rantehr than all n? Give the worst-case running time of selection sort in $\theta - notation$. Is the best-case run time any better?

```
# selection sort code (in place)
def selectionSort(array: list[int]) -> None:
```

```
MAX_INT: int = 1000000000

n: int = len(array)
i: int = 0

while (i < n-1):
    cur_min: int = MAX_INT
    min_i: int = 0

    for j in range(i, n):
        num: int = array[j]

        if (num < cur_min):
            cur_min = num
            min_i = j

    temp: int = array[i]
    array[i] = array[min_i]
    array[min_i] = temp
    i += 1
```

The loop invariant that this algorithm maintains (the property that holds true before and after every iteratoin of the loop). A[0:i-1] contains the i-1 smallest elements of the array and they are in sorted order

The algorithm only needs to run for the first n-1 elements because of the way it works it selects the smallest element for the current subarray being checked A[i:n] it then places that in the lowest postion in subarray and does this $\forall i \in [0 : n-1]$ so by the time i gets to n-1 this sub array is A[n-1:n] which only contains 1 element and thus is already sorted.

The worst case runtime of selection sort would be $\theta(n^2)$

The best case runtime of selection sort would be $\theta(n^2)$ which is the same. There is no mechanism to check if the subarray being checked is already sorted.

2.2-3

Consider linear search again (see Exercise 2.1-4). How many elements of the input array need to be checked on the average, assuming that the element being searched for is equally likely to be any element in the array? How about in the worst case? Using $\theta - notation$, give the average-case and worst-case run times of linear search. Justify your answers.

Let x be the element we are searching for. Let n be the number of elements in the array.

Assuming that x exists in the array then statistically on average we should have to check roughly n/2 elements to find x. Since x is equally likely to be at each index the average would look like $\frac{(1+2+...+n)}{n}$ and we know $1+2+...+n = \frac{n(n+1)}{2}$ thus using substitution we get an average runtime of $\frac{n(n+1)}{2n} = \frac{n+1}{2} = \frac{n}{2} + \frac{1}{2}$

In the worst cast the element would be at the last index in the array so at A[n-1] in this scenerio we have to check all n elements to find x.

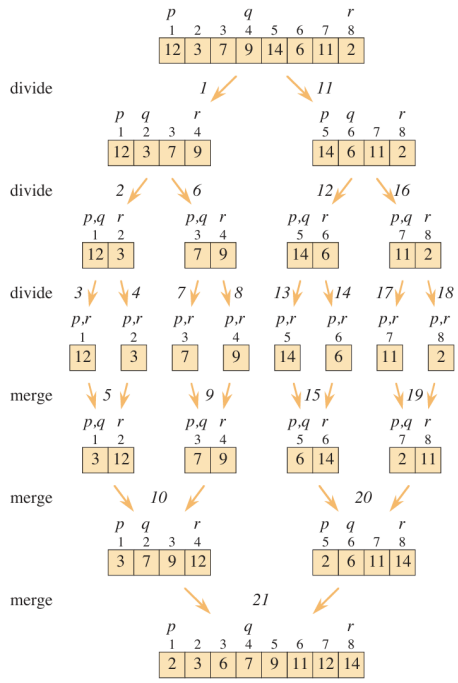
Notice: the dominant term for both the average and worst case remains $\theta(n)$ the only thing that changes is the constants.

2.2-4
How can you modify any sorting algorithm to have a good best-case running time?
You can add a way to check if the subarray you are currently finding the min for is already sorted. so maybe somthing like this

```
# selection sort code (in place)
def selectionSort(array: list[int]) -> None:

    n: int = len(array)
    i: int = 0

    while (i < n-1):
        cur_min: int = array[i]
        min_i: int = 0
        already_sorted: bool = True

        for j in range(i, n):
            num: int = array[j]

            if (j != n-1 and array[j + 1] < num):
                already_sorted = False

            if (num < cur_min):
                cur_min = num
                min_i = j

        if (already_sorted):
            return

        temp: int = array[i]
        array[i] = array[min_i]
        array[min_i] = temp
        i += 1
```

Section 2.3 (Designing algorithms)

2.3-1
Using Figure 2.4 as a model, illustrate the operation of merge sort on an array initially containing the sequence $< 3, 41, 52, 26, 38, 57, 9, 49 >$.



Chapter 3 (Characterizing Running Times)

Chapter 4 (Divide-and-Conquer)

Chapter 5 (Probabilistic Analysis and Randomized Algorithms)

Chapter 6 (Heapsort)

Chapter 7 (Quicksort)

Chapter 8 (Sorting in Linear Time)

Chapter 9 (Medians and Order Statistics)

Chapter 10 (Elementary Data Structures)

Chapter 11 (Hash Tables)

Chapter 12 (Binary Search Trees)

Chapter 13 (Red-Black Trees)

Chapter 14 (Dynamic Programming)

Chapter 15 (Greedy Algorithms)

Chapter 16 (Amortized Analysis)

Chapter 17 (Augmenting Data Structures)

Chapter 18 (B-Trees)

Chapter 19 (Data Structures for Disjoint Sets)

Chapter 20 (Elementary Graph Algorithms)

Chapter 21 (Minimum Spanning Trees)

Chapter 22 (Single-Source Shortest Paths)

Chapter 23 (All-Pairs Shortest Paths)

Chapter 24 (Maximum Flow)

Chapter 25 (Matchings in Bipartite Graphs)

Chapter 26 (Parallel Algorithms)

Chapter 27 (Online Algorithms)

Chapter 28 (Matrix Operations)

Chapter 29 (Linear Programming)

Chapter 30 (Polynomials and the FFT)

Chapter 31 (Number-Theoretic Algorithms)

Chapter 32 (String Matching)

Chapter 33 (Machine-Learning Algorithms)

Chapter 34 (NP-Completeness)

Chapter 35 (Approximation Algorithms)